Week 8, October 28, 2026

Main topics

- Iterative solutions for inverse problems
- Tomography at large scale

Material P. C. Hansen, *Discrete Inverse Problems: Insight and Algorithms*, Chapter 6 supplemented by Kirsch, section 2.6.
No new material.

Program

13:00-13:30 Iterative methods: Landweber, CGLS, projected Landweber

13:40-15:40 Exercises, see below

15:50-17:00 Research lecture: **Computed tomography of a Martian Meteorite** by Martin Sæbye Carøe, PhD student, DTU Compute.

Exercises

1. **How large is large?** Iterative methods are relevant for large scale problems, but how large is large? Take $N = 2^n$ for $n = 7, 8, 9, \ldots$ and generate a random matrix $K \in \mathbb{R}^{N \times N}$ and a random vector y.

   (a) How large an N can you handle if you want to compute the SVD in a few seconds? How does the time grow when N is doubled?

   (b) A dense $N \times N$ matrix in double precision takes $8N^2$ bytes. For which N does K no longer fit in the memory of your computer?

   (c) An image with $m \times m$ pixels has $N = m^2$ unknowns. How much memory does the dense matrix of a linear operator on the image need for $m = 64, 128, 256, 512$? What do you conclude about the way we must implement K for images?

2. **Landweber's method on the Heat problem.** Landweber's method is

$$x^{[k]} = x^{[k-1]} + \omega\, K^T\big(y^\delta - K x^{[k-1]}\big), \qquad x^{[0]} = 0,$$

   and it converges for $0 < \omega < 2/\|K\|_2^2$. Use the 1D Heat problem with Dirichlet boundary conditions (from week 1-2), i.e. K is the integral operator on $(0, \pi)$ with kernel $k(t, s) = \frac{2}{\pi} \sum_{n \geq 1} e^{-n^2 T} \sin(ns) \sin(nt)$. It is symmetric with singular values $\mu_i = e^{-i^2 T}$, so $\|K\|_2 = \mu_1$. Set up the matrix K with $n = 64$, $T = 0.01$ and the exact solution $x(s) = \min(s, \pi - s)/(\pi/2) + 0.6 \cdot \mathbf{1}_{\{|s-\pi/2|<0.3\}}$. Compute the right hand side $y = Kx$ and add Gaussian

noise d with $\|d\|_2 = \delta = 0.01\|y\|_2$ (1% relative noise), so $y^\delta = y + d$. The relative error of an iterate is $\|x^{[k]} - x\|_2/\|x\|_2$.

(a) Run the Landweber iteration and store the relative error history. Use $\omega = 1/\|K\|_2^2$ and run up to $k = 5000$ iterations. Plot the error history (log scale on both axes). Do you see semi-convergence? At which iteration is the error smallest, and what is the smallest error?

(b) *Experiment with the step length.* Repeat with $\omega = c/\|K\|_2^2$ for $c = 0.25,\ 0.5,\ 1,\ 1.5,\ 1.9$ and also for $c = 2.05$. Tabulate for each c the best iteration number k_{opt} and the smallest error. What happens to the *quality* of the best solution, and what happens to *when* you reach it? What happens for $c > 2$? Can you explain your observations from the filter factors?

(c) Repeat with noise-free data and $\omega = 1.9/\|K\|_2^2$. How many iterations do you need? What does this tell you about Landweber's method?

(d) Revisit the plot of filter factors from exercise 1 in week 4 (TSVD and Tikhonov). With $\mu_i = e^{-i^2 T}$, add the filter factors of Landweber, $f_i^{[k]} = 1 - (1 - \omega\mu_i^2)^k$, for $k = 10, 100, 1000$. Find Tikhonov parameters α that give similar filter factors. Which relation between $k\omega$ and α can you find?

3. **CGLS on the Abel problem.** CGLS computes $x^{[k]}$ as the minimizer of $\|Kx - y^\delta\|_2$ over the Krylov subspace $\mathcal{K}_k = \text{span}\{K^T y^\delta, \ldots, (K^T K)^{k-1} K^T y^\delta\}$. Start with $x^{[0]} = 0$, $r^0 = y^\delta$, $s^0 = K^T r^0$, $d^0 = s^0$ and iterate for $k = 1, 2, \ldots$

$$a_k = \frac{\|s^{k-1}\|^2}{\|Kd^{k-1}\|^2}, \quad x^{[k]} = x^{[k-1]} + a_k d^{k-1}, \quad r^k = r^{k-1} - a_k K d^{k-1}, \quad s^k = K^T r^k,$$

$$b_k = \frac{\|s^k\|^2}{\|s^{k-1}\|^2}, \qquad d^k = s^k + b_k d^{k-1}.$$

(`lsqr` is a robust implementation of the same iterates, but to record the error history you should write the loop yourself.) Use the Abel problem, $(Kx)(t) = \int_0^t x(s)/\sqrt{t-s}\,\mathrm{d}s$ for $t \in [0, 1]$, with $n = 64$ and the exact solution $x(s) = s^2\cos(\pi s/2)$. Noise is added as in exercise 2, now with the relative noise level $\delta/\|y\|_2$ stated in each item.

(a) Implement CGLS. Run it with noise levels 1%, 5% and 20% and plot the three error histories. At which iteration is the error smallest in each case? Plot a few iterates ($k = 1, 2, 3, 6, 15$) together with the exact solution.

(b) The Abel problem is only mildly ill-posed: compute $\text{cond}(K)$ for $n = 64$. How does this explain that you need quite a lot of noise to see semi-convergence?

(c) Run Landweber's method with $\omega = 1.5/\|K\|_2^2$ on the same data. Compare with CGLS: the number of iterations needed and the smallest error.

(d) Now run CGLS and Landweber (with $\omega = 1/\|K\|_2^2$) on the Heat problem with 1% noise. Compare the best iteration numbers. Can you explain the difference compared to Abel?

(e) Consider Figure 6.6 in *Discrete Inverse Problems: Insight and Algorithms*. Take the Abel model problem and do a similar analysis.

4. **Projected Landweber.** We illustrate the use of Landweber's iteration with a projection,

$$x^{[k]} = P\Big(x^{[k-1]} + \omega\, K^T\big(y^\delta - Kx^{[k-1]}\big)\Big),$$

where P represents a nonnegativity constraint: if x and $z = P(x)$ are vectors, then $z_i = x_i$ for $x_i \geq 0$ and $z_i = 0$ for $x_i < 0$. This method converges to a non-negative least squares solution. We use the Abel problem with $n = 64$, $\omega = 1.5/\|K\|_2^2$, and the non-negative function $x(s) = s^2\cos(\pi s/2)$.

(a) Implement the standard and the projected version. Run both with **no noise**; you may need several hundred iterations. Compute the error histories and comment.

(b) Add noise at the 1% and 10% level and run again. Do you see semi-convergence for both methods? At which iteration is the smallest error obtained, and how large is it? What happens for large k?

(c) Repeat (b) with the non-negative function that is exactly zero on parts of the interval,

$$\begin{aligned} x(s) &= \sin\big(\pi(s-0.3)/0.4\big) && \text{for } 0.3 < s < 0.7, \\ x(s) &= \tfrac{1}{2}\sin\big(\pi(s-0.75)/0.2\big) && \text{for } 0.75 < s < 0.95, \end{aligned}$$

and $x(s) = 0$ elsewhere. Plot the solutions for both methods at the best k and for a large k. Where does the projection help, and why?

5. **A large 2D deblurring problem.** (This problem is also the starting point for the miniproject 2, so write your code so that you can reuse it.) The forward heat problem on $(0,\pi)^2$ with homogeneous Dirichlet boundary conditions maps an image $x(s_1, s_2)$ to a blurred image $y(t_1, t_2) = \int k(t,s)x(s)\,\mathrm{d}s$ with kernel

$$k(t,s) = \Big(\frac{2}{\pi}\Big)^2 \sum_{k,l\geq 1} e^{-(k^2+l^2)T}\, \sin(ks_1)\sin(ls_2)\, \sin(kt_1)\sin(lt_2).$$

For an image $x \in \mathbb{R}^{m\times m}$ (pixel centers $s_j = (j - \frac{1}{2})\pi/m$) compute a discretization of $y = Kx$ by a 2D fast sine transform, multiplying by the singular values, and transforming back (e.g. using `scipy.fft.dstn` and `idstn` with `type=2, norm='ortho'`). Seen as a matrix K acting on $x = \mathrm{vec}(X)$, K is $m^2 \times m^2$, symmetric, and has singular values $e^{-(k^2+l^2)T}$, but **we never form it**. Use e.g. $T = 10^{-3}$. Implement a function that approximates $K(x)$. Test it for $m = 64, 128, 256, 512$, and check that it works on some well-chosen images x.